

Arbitrary Precision Arithmetic Library - Project Report

Name: Asvinth (ROLL NO: ES23BTECH11009)

May 2, 2025

Design Decisions

The project is structured to cleanly separate logic, build, and interface:

- Java is used to implement core logic due to the need for precise handling of large numbers.
- Operations are separated into two main classes: `AInteger` for integers and `AFloat` for floating-point numbers, respectively.
- The entry point `MyInfArith.java` is responsible for input parsing and dispatching to the correct class.
- Python script `run_arithmetic.py` serves as a user-friendly interface to compile and run the code with a single command.
- Ant (`build.xml`) is used for standardized compilation and JAR creation.

This separation allows for easier testing, maintenance, and expansion (e.g., adding complex number support).

README (Usage Instructions)

To use this library, follow the steps below:

1. Build the Project

```
ant build
```

2. Run an Operation Using the Python Script

```
python run_arithmetic.py <int/float> <operation> <operand1> <operand2>
```

Examples:

```
python run_arithmetic.py int add 123456789 987654321
python run_arithmetic.py float div 22.5 7.5
```

3. Run an Operation Using the compiled `aarithmetic.jar` file

```
java -jar aarithmetics.jar <int/float> <add/sub/mul/div> <operand1> <operand2>
```

Examples:

```
java -jar aarithmetic.jar float div 10 6
java -jar aarithmetic.jar float mul 56.4534 74.345
```

3. Operations Supported

`add`, `sub`, `mul`, `div`

4. Modes Supported

`int` for Integer arithmetic `float` for Floating-point arithmetic

Limitations

- The project does not handle user input through GUI or file-based interfaces.
- No graceful error messages are shown to the user (e.g., division by zero).
- The float division precision is fixed to 30 rounding defaults.
- Exceptions are raised directly instead of being caught and presented more cleanly.
- No unit test framework is integrated (testing is manual via CLI).

Verification Approach

- Manual testing was conducted by running various test cases through the CLI.
- Edge case test scenarios included:
 - Division by zero (to check exception handling)
 - Operations with very large numbers (to test the limits)
 - Negative values and combinations of positive/negative operands
 - Invalid operations or modes (e.g., `floatt`, `ad`)
- Examples:

```
int div 10 0          # Should throw terminal error
float add 1.123456789123456789 9.876543210876543210 # Precision test
int sub -9999999999999999 1 # Large number subtraction
```

Key Learnings

- Learned how to use Ant for Java builds and organizing project structure.
- Understood how to use Python `subprocess` to interact with Java programs.
- Learned how `BigInteger` and `BigDecimal` work handling arbitrary precision and implements.
- Gained experience in documenting and modularizing a software library.